Fig. 2.15 Comparison.java

```java
//~~~~ Fig. 2.15 COMPARISON ~~~~ //
import java.util.Scanner;

public class Comparison {
    // metode main memulai eksekusi aplikasi Java
    public static void main(String[] args) {
        // buat Scanner untuk memperoleh input dari baris perintah
        Scanner input = new Scanner(System.in);

        //----- NAMA & NPM -----//
        System.out.println(" --NAMA-------------------------");
        System.out.println(" | MUHAMMAD FEBRI ARIDAMAYANTO |");
        System.out.println(" -------------------------------");

        System.out.println(" --NPM--------");
        System.out.println(" | 262310005 |");
        System.out.println(" -------------");

        System.out.print("Masukkan bilangan bulat pertama: "); //prompt
        int number1 = input.nextInt(); // membaca bilangan bulat pertama dari pengguna

        System.out.print("Masukkan bilangan bulat kedua: "); //prompt
        int number2 = input.nextInt(); // membaca bilangan bulat kedua dari pengguna

        //--- Contoh Java kondisi & Makna kondisi dalam Java ---//
        // X == Y ----> X sama dengan Y
        // X != Y ----> X tidak sama dengan Y
        // X < Y -----> X lebih kecil dari Y
        // X > Y -----> X lebih besar dari Y
        // X <= Y ----> X lebih kecil atau sama dengan Y
        // X >= Y ----> X lebih besar atau sama dengan Y
        //-------------------------------------------------------//

        if (number1 == number2)
            System.out.printf("%d == %d%n", number1, number2);

        if (number1 != number2)
            System.out.printf("%d != %d%n", number1, number2);

        if (number1 < number2)
            System.out.printf("%d < %d%n", number1, number2);

        if (number1 > number2)
            System.out.printf("%d > %d%n", number1, number2);

        if (number1 <= number2)
            System.out.printf("%d <= %d%n", number1, number2);

        if (number1 >= number2)
            System.out.printf("%d >= %d%n", number1, number2);

        input.close(); // menutup Scanner
    }
}
```

Output Comparison.java

```
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>  & 'C:\Users\pebri\.vscode\extensions\redhat.java-1.57.0-win32-x
64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pebri\OneDrive\D
esktop\Logical Thinking\bin' 'Comparison'
 -NAMA-----------------------
| MUHAMMAD FEBRI ARIDAMAYANTO |
 -----------------------------

 -NPM---------
| 262310005 |
 -------------

Masukkan bilangan bulat pertama: 666
Masukkan bilangan bulat kedua: 666
666 == 666
666 <= 666
666 >= 666
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking> ^C
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>  c:; cd 'c:\Users\pebri\OneDrive\Desktop\Logical Thinking'; & 'C
:\Users\pebri\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeD
etailsInExceptionMessages' '-cp' 'C:\Users\pebri\OneDrive\Desktop\Logical Thinking\bin' 'Comparison'
 -NAMA-----------------------
| MUHAMMAD FEBRI ARIDAMAYANTO |
 -----------------------------

 -NPM---------
| 262310005 |
 -------------

Masukkan bilangan bulat pertama: 4000
Masukkan bilangan bulat kedua: 5000
4000 != 5000
4000 < 5000
4000 <= 5000
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking> ^C
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>  c:; cd 'c:\Users\pebri\OneDrive\Desktop\Logical Thinking'; & 'C
:\Users\pebri\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeD
etailsInExceptionMessages' '-cp' 'C:\Users\pebri\OneDrive\Desktop\Logical Thinking\bin' 'Comparison'
 -NAMA-----------------------
| MUHAMMAD FEBRI ARIDAMAYANTO |
 -----------------------------

 -NPM---------
| 262310005 |
 -------------

Masukkan bilangan bulat pertama: 3000
Masukkan bilangan bulat kedua: 1000
3000 != 1000
3000 > 1000
3000 >= 1000
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>
```

Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java Comparison di terminal Visual Studio Code. Program digunakan untuk membandingkan dua bilangan bulat menggunakan operator perbandingan, yaitu ==, !=, <, >, <=, dan >=.

Pada gambar terdapat tiga kali pengujian dengan nilai yang berbeda.

1. Pengujian pertama: bilangan 666 dan 666

- **666 == 666 → kondisi benar karena kedua bilangan bernilai sama.**
- **666 <= 666 → kondisi benar karena bilangan pertama sama dengan bilangan kedua.**
- **666 >= 666 → kondisi benar karena bilangan pertama sama dengan bilangan kedua.**

2. Pengujian kedua: bilangan 4000 dan 5000

- **4000 != 5000 → kondisi benar karena kedua bilangan berbeda.**
- **4000 < 5000 → kondisi benar karena 4000 lebih kecil dari 5000.**
- **4000 <= 5000 → kondisi benar karena 4000 lebih kecil dari 5000.**

3. Pengujian ketiga: bilangan 3000 dan 1000

- **3000 != 1000 → kondisi benar karena kedua bilangan berbeda.**
- **3000 > 1000 → kondisi benar karena 3000 lebih besar dari 1000.**
- **3000 >= 1000 → kondisi benar karena 3000 lebih besar dari 1000.**

Fig. 3.6 ClassAverage.java

```java
//~~~~ Fig. 3.6 CLASS AVERAGE ~~~~ //
import java.util.Scanner;

public class ClassAverage {
    public static void main(String[] args) {
        //buat Scanner untuk memperoleh input dari jendela perintah
        Scanner input = new Scanner(System.in);

        //----- NAMA & NPM -----//
        System.out.println(" --NAMA-------------------------");
        System.out.println(" | MUHAMMAD FEBRI ARIDAMAYANTO |");
        System.out.println(" -------------------------------");

        System.out.println(" --NPM--------");
        System.out.println(" | 262310005 |");
        System.out.println(" -------------");

        //tahap inisialisasi
        int total = 0; //jumlah dari nilai-nilai yang dimasukkan pengguna
        int gradeCounter = 1; //jumlah dari nilai-nilai yang dimasukkan pengguna

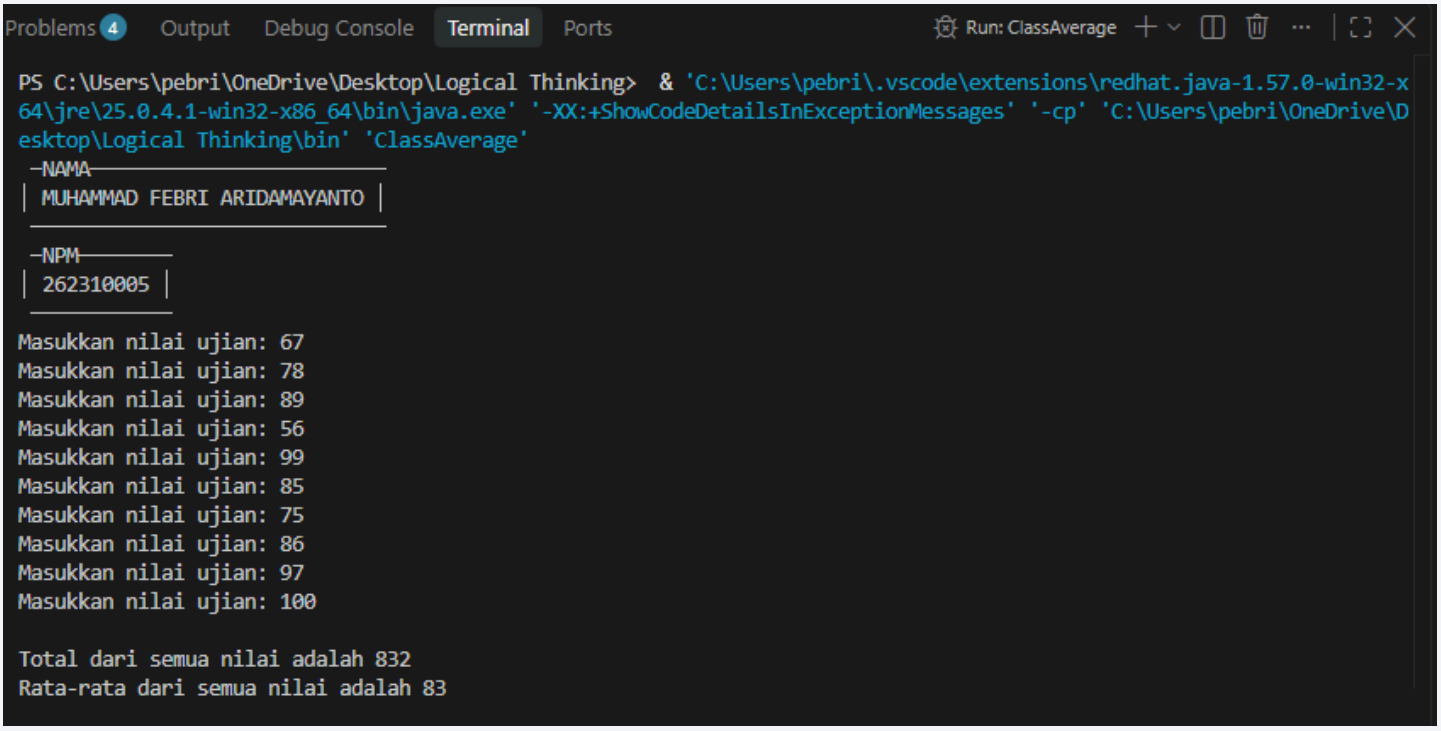
        //tahap pemrosesan menggunakan iterasi yang dikendalikan oleh pencacah
        while (gradeCounter <= 10) { //loop 10 kali
            System.out.print("Masukkan nilai ujian: "); //prompt
            int grade = input.nextInt(); //membaca nilai ujian
            total = total + grade; //menambahkan nilai ke total
            gradeCounter = gradeCounter + 1; //menambahkan 1 ke pencacah
        }

        //tahap terminasi
        int average = total / 10; //pembagian bilangan bulat menghasilkan
                                  //hasil berupa bilangan bulat

        //tampilkan total dan rata-rata nilai
        System.out.printf("%nTotal dari semua nilai adalah %d%n", total);
        System.out.printf("Rata-rata dari semua nilai adalah %d%n", average);

        input.close(); //menutup Scanner
    }
}
```

Output ClassAverage.java



Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java ClassAverage pada terminal Visual Studio Code. Program ini digunakan untuk menerima sepuluh nilai ujian, menghitung total seluruh nilai, dan menentukan nilai rata-rata kelas.

Pada awal program, ditampilkan identitas mahasiswa berupa nama MUHAMMAD FEBRI ARIDAMAYANTO dan NPM 262310005. Selanjutnya, pengguna memasukkan sepuluh nilai ujian, yaitu 67, 78, 89, 56, 99, 85, 75, 86, 97, dan 100.

Berdasarkan hasil eksekusi program, diperoleh total seluruh nilai sebesar 832. Nilai rata-rata dihitung dengan membagi total nilai dengan jumlah data, yaitu 832 dibagi 10, sehingga diperoleh rata-rata 83,2. Namun, pada output program ditampilkan sebagai 83.

Fig. 3.8 ClassAverage.java

```java
//~~~~ Fig. 3.8 CLASS AVERAGE ~~~~//
import java.util.Scanner;

public class ClassAverage2 {
    public static void main(String[] args) {
        //buat Scanner untuk memperoleh input dari jendela perintah
        Scanner input = new Scanner(System.in);

        //----- NAMA & NPM -----//
        System.out.println(" ---NAMA----------------------------");
        System.out.println(" |  MUHAMMAD FEBRI ARIDAMAYANTO  | ");
        System.out.println(" ---------------------------------");

        System.out.println(" ---NPM---------");
        System.out.println(" | 262310005  |";
        System.out.println(" --------------");

        //tahap inisialisasi
        int total = 0; //inisialisasi jumlah nilai
        int gradeCounter = 0; //inisialisasi jumlah nilai yang telah dimasukkan sejauh i

        //tahap pemrosesan
        //meminta masukan dan membaca nilai dari pengguna
        System.out.print("Masukkan nilai atau -1 untuk keluar: ");
        int grade = input.nextInt();

        //lakukan perulangan sampai nilai penanda dibaca dari pengguna
        while (grade != -1) {
            total = total + grade; //tambahkan nilai ke total
            gradeCounter = gradeCounter + 1; //menambah penghitung

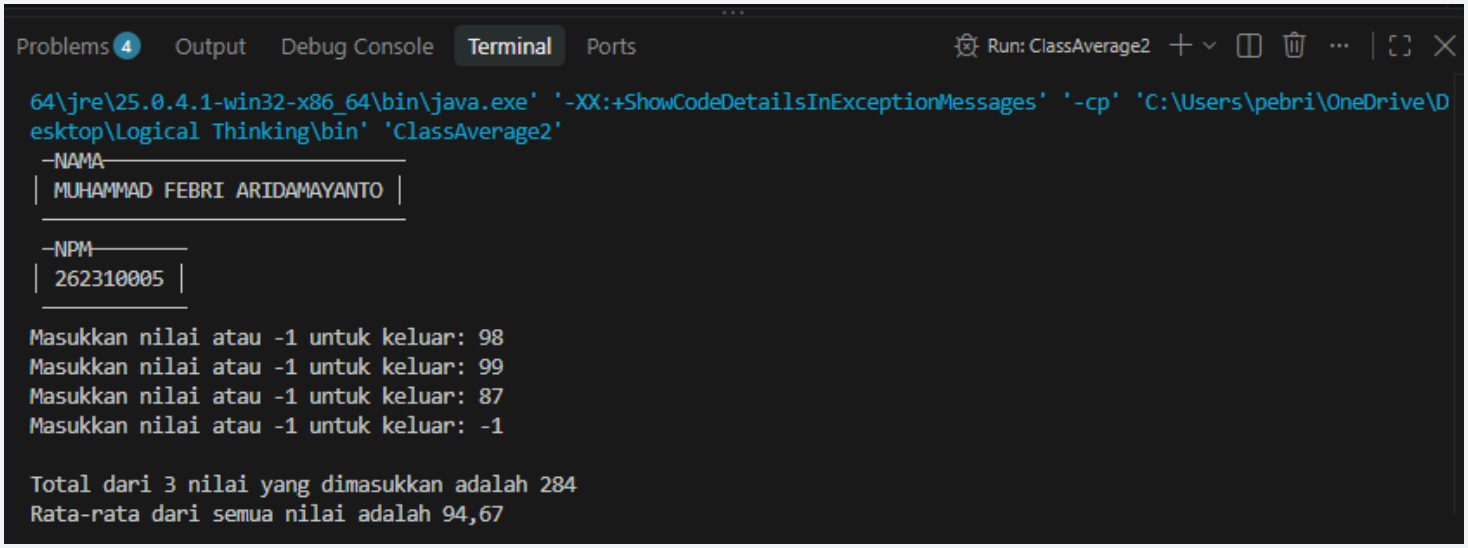
            //meminta masukan dan membaca nilai berikutnya dari pengguna
            System.out.print("Masukkan nilai atau -1 untuk keluar: ");
            grade = input.nextInt();
        }

        //tahap terminasi
        //jika pengguna memasukkan setidaknya satu nilai...
        if (gradeCounter != 0) {
            //gunakan angka dengan titik desimal untuk menghitung rata-rata nilai
            double average = (double) total / gradeCounter;

            //tampilkan total dan rata-rata (dengan dua digit presisi)
            System.out.printf("%nTotal dari %d nilai yang dimasukkan adalah %d%n",
                              gradeCounter, total);
            System.out.printf("Rata-rata dari semua nilai adalah %.2f%n", average);
        }
        else { // tidak ada nilai yang dimasukkan, jadi tampilkan pesan yang sesuai
            System.out.println("Tidak ada nilai yang dimasukkan");
        }

        input.close(); //menutup Scanner
    }
}
```

Output ClassAverage.java



Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java ClassAverage2 pada terminal Visual Studio Code. Program ini digunakan untuk memasukkan nilai secara berulang, menghitung total nilai, dan menentukan rata-rata dari nilai yang dimasukkan. Pengguna dapat mengakhiri proses input dengan memasukkan angka -1.

Pada awal program, ditampilkan identitas mahasiswa berupa nama MUHAMMAD FEBRI ARIDAMAYANTO dan NPM 262310005. Selanjutnya, pengguna memasukkan nilai 98, 99, dan 87. Setelah itu, pengguna memasukkan angka -1 sebagai tanda bahwa proses input telah selesai.

Berdasarkan hasil eksekusi, jumlah nilai yang dimasukkan adalah 3 nilai, dengan total keseluruhan sebesar 284. Nilai rata-rata dihitung dengan membagi total nilai dengan jumlah nilai, yaitu 284 dibagi 3, sehingga diperoleh rata-rata 94,67.

Kesimpulan: Program ClassAverage2 berhasil menerima input nilai secara berulang hingga pengguna memasukkan angka -1. Angka -1 berfungsi sebagai penanda untuk menghentikan input dan tidak ikut dihitung sebagai nilai ujian. Program kemudian menampilkan jumlah nilai yang dimasukkan, total seluruh nilai, dan rata-rata nilai secara otomatis.

Fig. 3.10 Analysis.java

```java
//~~~~ Fig. 3.10 ANALYSIS JAVA ~~~~//
import java.util.Scanner;

public class Analysis {
    public static void main(String[] args) {
        //buat Scanner untuk memperoleh input dari jendela perintah
        Scanner input = new Scanner(System.in);

        //----- NAMA & NPM -----//
        System.out.println(" --NAMA----------------------------");
        System.out.println(" | MUHAMMAD FEBRI ARIDAMAYANTO  | ");
        System.out.println(" ------------------------------");

        System.out.println(" ---NPM--------");
        System.out.println(" | 262310005  |");
        System.out.println(" --------------");

        //tahap inisialisasi
        int passes = 0; //jumlah siswa yang lulus
        int failures = 0; //jumlah siswa yang gagal
        int studentCounter = 1; //jumlah siswa yang sedang diproses

        //memproses 10 siswa menggunakan perulangan yang dikendalikan oleh pencacah
        while (studentCounter <= 10) {
            // meminta input dari pengguna dan mendapatkan nilai dari pengguna
            System.out.print("Masukkan hasil ujian (1 = lulus, 2 = gagal): ");
            int result = input.nextInt();

            //memeriksa apakah hasilnya adalah 1 atau 2
            if (result == 1) {
                passes = passes + 1; //menambahkan 1 ke jumlah siswa yang lulus
            }
            else {
                failures = failures + 1; //menambahkan 1 ke jumlah siswa yang gagal
            }

            //tambahkan nilai studentCounter agar perulangan pada akhirnya berhenti
            studentCounter = studentCounter + 1;
        }

        //tampilkan jumlah siswa yang lulus dan gagal
        System.out.printf("lulus: %d%nGagal: %d%n", passes, failures);

        //jika lebih dari 8 siswa lulus, tampilkan pesan tambahan
        if (passes > 8) {
            System.out.println("Selamat! Kelas ini sangat sukses!");
        }

        input.close(); //menutup Scanner
    }
}
```

Output Analysis.java

```
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>  & 'C:\Users\pebri\.vscode\extensions\redhat.java-1.57.0-win32-x
64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pebri\OneDrive\D
esktop\Logical Thinking\bin' 'Analysis'
 -NAMA-------------------------
| MUHAMMAD FEBRI ARIDAMAYANTO |
 ---------------------------

 -NPM--------
| 262310005 |
 ------------

Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 2
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
lulus: 9
Gagal: 1
Selamat! Kelas ini sangat sukses!
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking> ^C
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>  c:; cd 'c:\Users\pebri\OneDrive\Desktop\Logical Thinking'; & 'C
:\Users\pebri\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeD
etailsInExceptionMessages' '-cp' 'C:\Users\pebri\OneDrive\Desktop\Logical Thinking\bin' 'Analysis'
 -NAMA-------------------------
| MUHAMMAD FEBRI ARIDAMAYANTO |
 ---------------------------

 -NPM--------
| 262310005 |
 ------------

Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 2
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 2
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 2
Masukkan hasil ujian (1 = lulus, 2 = gagal): 2
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
Masukkan hasil ujian (1 = lulus, 2 = gagal): 1
lulus: 6
Gagal: 4
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>
```

Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java Analysis pada terminal Visual Studio Code. Program ini digunakan untuk menganalisis hasil ujian dengan menerima input berupa angka 1 untuk menyatakan lulus dan angka 2 untuk menyatakan gagal. Program menghitung jumlah peserta yang lulus dan gagal, kemudian menampilkan hasil analisisnya.

Pada bagian pertama, pengguna memasukkan hasil ujian sebanyak 10 kali, dengan 9 peserta lulus dan 1 peserta gagal. Program kemudian menampilkan jumlah kelulusan dan pesan "Selamat! Kelas ini sangat sukses!".

Pada bagian kedua, pengguna kembali memasukkan 10 hasil ujian. Dari data tersebut, diperoleh 6 peserta lulus dan 4 peserta gagal. Program menampilkan jumlah peserta yang lulus dan gagal tanpa menampilkan pesan keberhasilan kelas seperti pada pengujian pertama.

Kesimpulan: Program Analysis berhasil menghitung jumlah peserta yang lulus dan gagal berdasarkan data hasil ujian yang dimasukkan. Program juga dapat memberikan pesan tambahan ketika jumlah peserta yang lulus menunjukkan bahwa kelas tersebut sangat sukses. Hal ini menunjukkan penerapan variabel penghitung, perulangan, dan percabangan kondisi dalam bahasa pemrograman Java.

Fig. 3.13 Increment.java

```java
//~~~~ Fig. 3.13 INCREMENT ~~~~//
import java.util.Scanner;

public class Increment {
    public static void main(String[] args) {
        //buat Scanner untuk memperoleh input dari jendela perintah
        Scanner input = new Scanner(System.in);

        //----- NAMA & NPM ------//
        System.out.println(" ---NAMA-------------------------");
        System.out.println(" |  MUHAMMAD FEBRI ARIDAMAYANTO | ");
        System.out.println(" --------------------------------");

        System.out.println(" ---NPM---------");
        System.out.println(" |  262310005 | ");
        System.out.println(" ---------------");

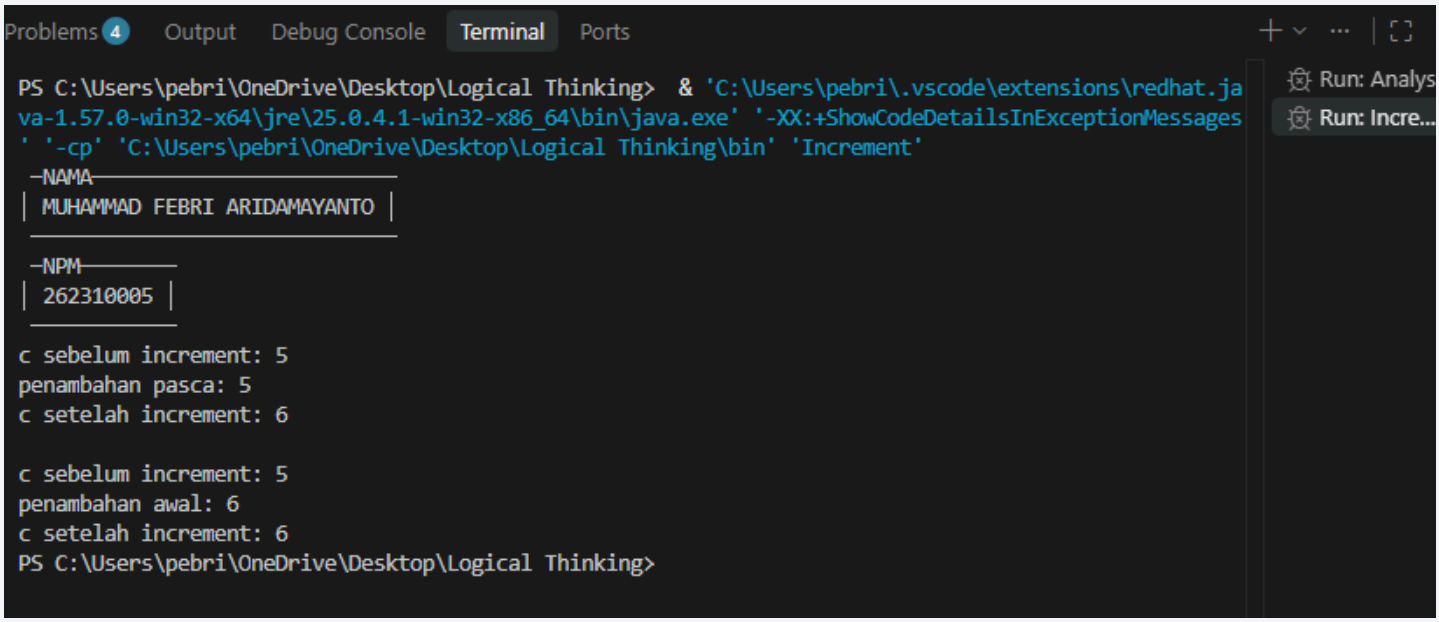
        //menunjukkan operator increment postfix
        int c = 5; //inisialisasi c
        System.out.printf("c sebelum increment: %d%n", c); //menampilkan nilai c
        System.out.printf("penambahan pasca: %d%n", c++); //menampilkan c lalu
                                //menambahkan 1 ke c
        System.out.printf("c setelah increment: %d%n", c); //menampilkan nilai c

        System.out.println(); //mencetak baris kosong

        //menunjukkan operator increment prefix
        c = 5; //menginisialisasi ulang c
        System.out.printf("c sebelum increment: %d%n", c); //menampilkan nilai c
        System.out.printf("penambahan awal: %d%n", ++c); //menambahkan 1 ke c lalu
                                //menampilkan c
        System.out.printf("c setelah increment: %d%n", c); //menampilkan nilai c

        input.close(); //menutup Scanner
    }
}
```

Output Increment.java



Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java Increment pada terminal Visual Studio Code. Program ini digunakan untuk menunjukkan perbedaan antara operator post-increment (c++) dan pre-increment (++c) dalam bahasa pemrograman Java.

Pada bagian pertama, nilai awal variabel c adalah 5. Ketika operator post-increment digunakan, nilai c ditampilkan terlebih dahulu, kemudian bertambah menjadi 6. Oleh karena itu, output menunjukkan nilai sebelum increment sebesar 5, penambahan pasca sebesar 5, dan nilai setelah increment sebesar 6.

Pada bagian kedua, nilai awal variabel c kembali menjadi 5. Ketika operator pre-increment digunakan, nilai c bertambah terlebih dahulu menjadi 6, kemudian nilai tersebut digunakan dalam proses penampilan output. Hasilnya, penambahan awal dan nilai setelah increment sama-sama menunjukkan angka 6.

Kesimpulan: Program Increment menunjukkan bahwa operator post-increment (c++) menggunakan nilai variabel terlebih dahulu sebelum menaikkannya, sedangkan operator pre-increment (++c) menaikkan nilai variabel terlebih dahulu sebelum menggunakannya. Keduanya menambah nilai variabel sebesar satu, tetapi memiliki urutan pelaksanaan yang berbeda.

Fig. 4.1 WhileCounter.java

```java
//~~~~ Fig. 4.1 WHILE COUNTER ~~~~~//
public class WhileCounter {
    public static void main(String[] args) {

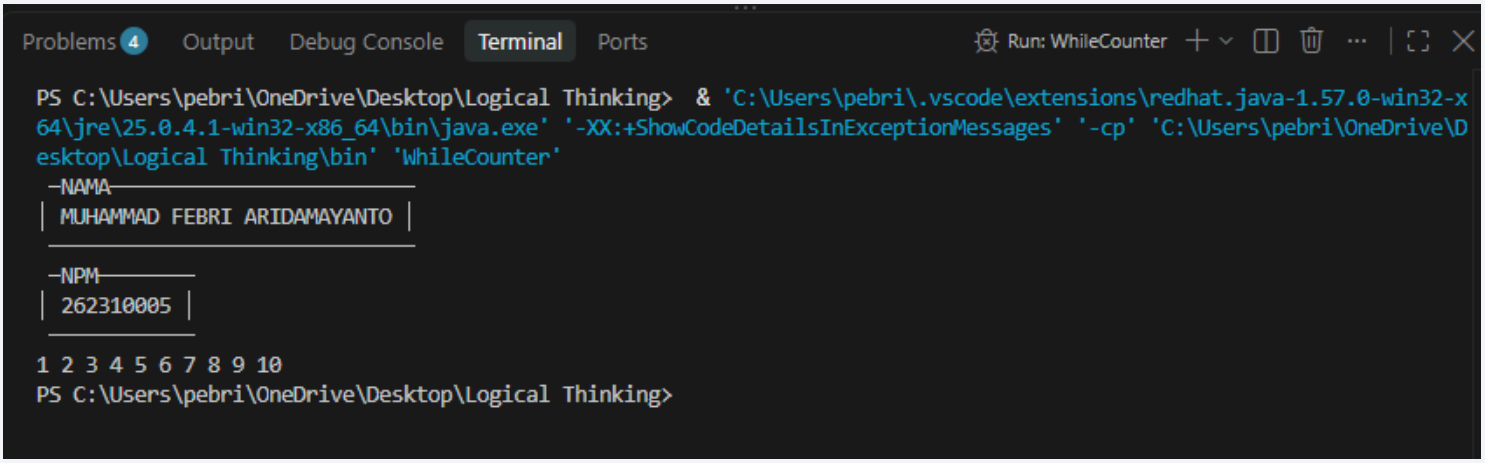
        //----- NAMA & NPM -----//
        System.out.println(" —NAMA--------------------------");
        System.out.println(" | MUHAMMAD FEBRI ARIDAMAYANTO |");
        System.out.println(" -------------------------------");

        //----- NPM---------
        System.out.println(" —NPM----------");
        System.out.println(" | 262310005 |");
        System.out.println(" ---------------");

        int counter = 1; //deklarasikan dan inisialisasi variabel kontrol

        while (counter <= 10) { //kondisi kelanjutan perulangan
            System.out.printf("%d ", counter); //menampilkan nilai counter
            ++counter; //increment counter
        }
        System.out.println();
    }
```

Output WhileCounter.java



Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java WhileCounter pada terminal Visual Studio Code. Program ini menggunakan struktur perulangan while untuk menampilkan urutan angka dari 1 sampai 10.

Pada awal program, ditampilkan identitas mahasiswa berupa nama MUHAMMAD FEBRI ARIDAMAYANTO dan NPM 262310005. Selanjutnya, program menampilkan deretan angka berikut: 1 2 3 4 5 6 7 8 9 10

Angka-angka tersebut ditampilkan secara berurutan karena perulangan dijalankan selama kondisi yang ditentukan masih bernilai benar. Pada setiap iterasi, nilai pencacah bertambah satu hingga mencapai angka 10. Setelah kondisi perulangan tidak lagi terpenuhi, program berhenti dan terminal kembali menampilkan prompt perintah.

Kesimpulan: Program WhileCounter berhasil menampilkan angka 1 sampai 10 menggunakan perulangan while. Program ini menunjukkan penggunaan variabel pencacah, kondisi perulangan, dan penambahan nilai secara bertahap dalam bahasa pemrograman Java.

Fig. 4.6 Interest.java

```java
//~~~~ Fig. 4.6 INTEREST ~~~~//
public class Interest {
    public static void main(String[] args) {

        //----- NAMA & NPM -----//
        System.out.println(" ---NAMA--------------------------");
        System.out.println(" | MUHAMMAD FEBRI ARIDAMAYANTO | ");
        System.out.println(" -------------------------------");

        //----- NPM--------
        System.out.println(" --NPM---------");
        System.out.println(" | 262310005 |");
        System.out.println(" -------------");

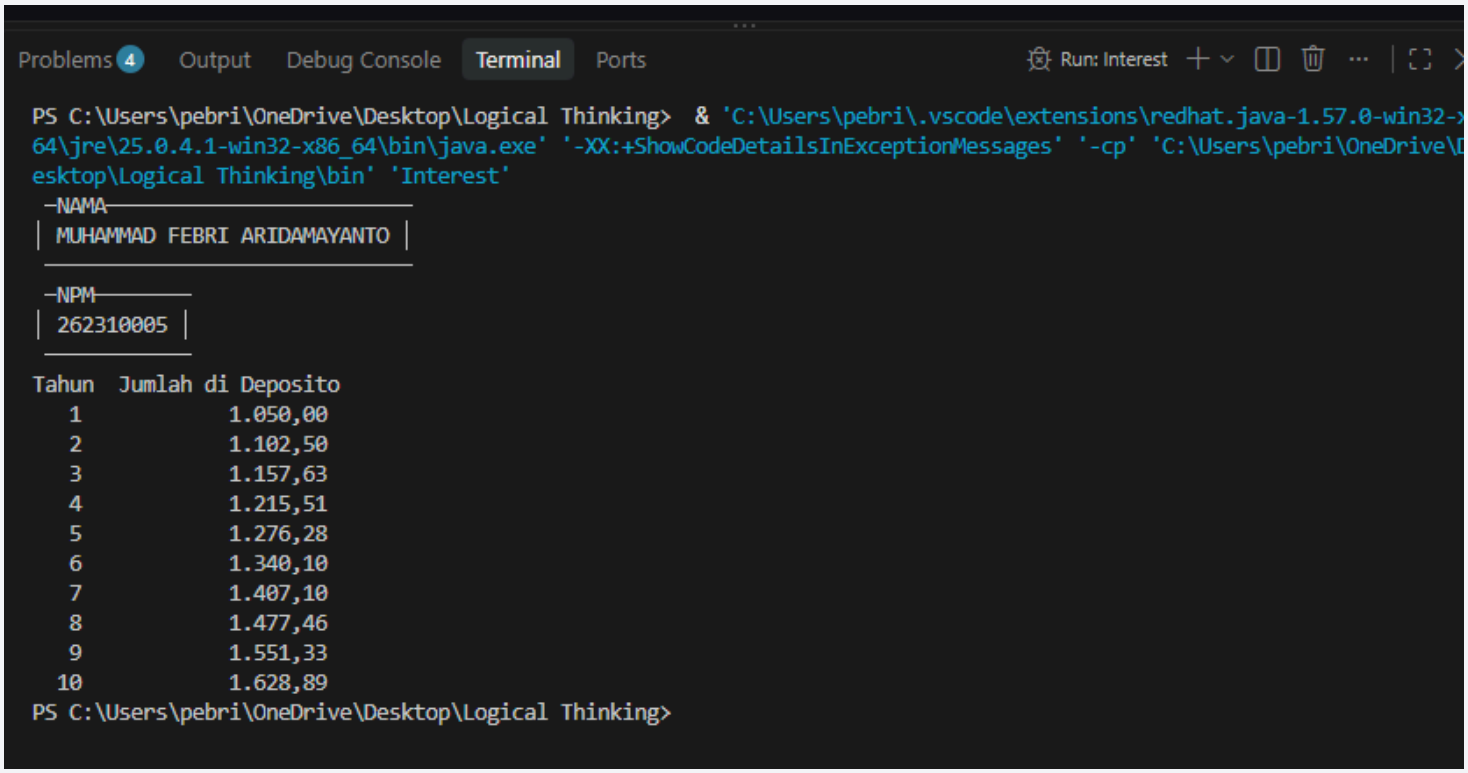
        double principal = 1000.0; //jumlah awal sebelum bunga
        double rate = 0.05; //tingkat bunga

        //tampilkan tabel judul
        System.out.printf("%s%20s%n", "Tahun", "Jumlah di Deposito");

        //tampilkan jumlah di deposito untuk setiap tahun
        for (int year = 1; year <= 10; ++year) {
            //hitung jumlah di deposito setelah year tahun
            double amount = principal * Math.pow(1.0 + rate, year);

            //tampilkan tahun dan jumlah di deposito
            System.out.printf("%4d%,20.2f%n", year, amount);
        }
    }
}
```

Output Interest.java



Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java Interest pada terminal Visual Studio Code. Program ini digunakan untuk menghitung pertumbuhan jumlah uang dalam deposito selama 10 tahun berdasarkan tingkat bunga yang telah ditentukan.

Pada awal program, ditampilkan identitas mahasiswa berupa nama MUHAMMAD FEBRI ARIDAMAYANTO dan NPM 262310005. Selanjutnya, program menampilkan tabel yang terdiri dari dua kolom, yaitu Tahun dan Jumlah di Deposito.

Berdasarkan hasil output, jumlah deposito pada tahun pertama adalah 1.050,00, kemudian meningkat menjadi 1.102,50 pada tahun kedua. Peningkatan jumlah deposito terus berlangsung setiap tahun hingga mencapai 1.628,89 pada tahun kesepuluh.

Hasil tersebut menunjukkan bahwa jumlah deposito bertambah secara bertahap karena adanya perhitungan bunga setiap tahun. Semakin lama uang disimpan, semakin besar pula jumlah deposito yang terkumpul.

Kesimpulan: Program Interest berhasil menampilkan perkembangan jumlah deposito selama 10 tahun. Program ini menunjukkan penerapan perulangan untuk menghitung pertumbuhan nilai uang dari tahun ke tahun serta penggunaan operasi aritmetika dalam bahasa pemrograman Java.

Fig. 4.9 LetterGrades.java

```java
//~~~~ Fig. 4.9 LETTER GRADES ~~~~//

import java.util.Scanner;

public class LetterGrades {
    public static void main(String[] args) {

        //----- NAMA & NPM ------//
        System.out.println("  --NAMA--------------------------------");
        System.out.println("   |  MUHAMMAD FEBRI ARIDAMAYANTO     |  ");
        System.out.println("-------------------------------------");

        System.out.println("  --NPM---------");
        System.out.println(" |  262310005   |");
        System.out.println("----------------");

        //tahap inisialisasi
        int total = 0; //jumlah dari nilai-nilai yang dimasukkan pengguna
        int gradeCounter = 1;  //jumlah dari nilai-nilai yang dimasukkan pengguna
        int aCount = 0;  //jumlah A
        int bCount = 0;  //jumlah B
        int cCount = 0;  //jumlah C
        int dCount = 0;  //jumlah D
        int fCount = 0;  //jumlah F

        Scanner input = new Scanner(System.in);

        System.out.printf("%s%n%s%n%n %s%n  %s%n",
                "Masukkan nilai ujian antara 0 dan 100.",
                "Ketik penanda akhir berkas untuk mengakhiri masukan.",
                "Pada UNIX/Linux/macOS, ketik <Ctrl> d lalu tekan Enter",
                "Pada Windows, ketik <Ctrl> z lalu tekan Enter");

        //lakukan perulangan sampai pengguna memasukkan penanda akhir berkas (end-of-file)
        while (input.hasNext()) {
            int grade = input.nextInt(); //membaca nilai ujian
            total += grade;  //menambahkan nilai ke total
            ++gradeCounter;  //menambahkan 1 ke pencacah

            //menentukan nilai huruf yang sesuai dengan nilai ujian
            switch (grade / 10) {
                case 9:  //nilai antara 90 dan 99
                case 10:  //nilai 100
                    ++aCount;  //menambahkan 1 ke jumlah A
                    break; //keluar dari switch
                case 8:  //nilai antara 80 dan 89
                    ++bCount;  //menambahkan 1 ke jumlah B
                    break; //keluar dari switch
                case 7:  //nilai antara 70 dan 79
                    ++cCount;  //menambahkan 1 ke jumlah C
                    break; //keluar dari switch
                case 6:  //nilai antara 60 dan 69
                    ++dCount;  //menambahkan 1 ke jumlah D
                    break; //keluar dari switch
                default: //nilai kurang dari 60
                    ++fCount;  //menambahkan 1 ke jumlah F
                    break; //keluar dari switch

            }
        }
        //tampilkan laporan nilai
        System.out.printf("%ngrade Report:%n");
        //jika pengguna memasukkan setidaknya satu nilai...
        if (gradeCounter != 0) {
            double average = (double) total / gradeCounter; //menggunakan angka dengan titik desimal untuk menghitung rata-rata nilai

            System.out.printf("Total dari %d nilai yang dimasukkan adalah %d%n", gradeCounter, total);
            System.out.printf("Rata-rata dari semua nilai adalah %.2f%n", average);
            System.out.printf("%nJumlah dari setiap nilai adalah:%n");
            System.out.printf("Jumlah A: %d%n", aCount);
            System.out.printf("Jumlah B: %d%n", bCount);
            System.out.printf("Jumlah C: %d%n", cCount);
            System.out.printf("Jumlah D: %d%n", dCount);
            System.out.printf("Jumlah F: %d%n", fCount);
        } else { // tidak ada nilai yang dimasukkan, jadi tampilkan pesan yang sesuai
            System.out.println("Tidak ada nilai yang dimasukkan");
        }

        input.close(); //menutup Scanner
    }
}
```

Output LetterGrades.java

```
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>  & 'C:\Users\pebri\.vscode\extensions\redhat.java-1.57.0-
64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pebri\One
esktop\Logical Thinking\bin' 'LetterGrades'
 -NAMA-------------------------
| MUHAMMAD FEBRI ARIDAMAYANTO |
 ---------------------------

 -NPM--------
| 262310005 |
 ------------
Masukkan nilai ujian antara 0 dan 100.
Ketik penanda akhir berkas untuk mengakhiri masukan.
 Pada UNIX/Linux/macOS, ketik <Ctrl> d lalu tekan Enter
  Pada Windows, ketik <Ctrl> z lalu tekan Enter
99
98
76
65
98
76
66
78
66
85
^Z

grade Report:
Total dari 11 nilai yang dimasukkan adalah 807
Rata-rata dari semua nilai adalah 73,36

Jumlah dari setiap nilai adalah:
Jumlah A: 3
Jumlah B: 1
Jumlah C: 3
Jumlah D: 3
Jumlah F: 0
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>
```

Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java LetterGrades pada terminal Visual Studio Code. Program ini digunakan untuk menerima nilai ujian, menghitung total dan rata-rata nilai, serta mengelompokkan nilai ke dalam kategori huruf A, B, C, D, dan F.

Pada awal program, ditampilkan identitas mahasiswa berupa nama MUHAMMAD FEBRI ARIDAMAYANTO dan NPM 262310005. Selanjutnya, pengguna memasukkan 11 nilai ujian, yaitu 99, 98, 76, 65, 98, 76, 66, 78, 66, 85, dan 0. Tanda ^Z digunakan untuk mengakhiri input pada Windows.

Berdasarkan hasil eksekusi, program menampilkan total nilai sebesar 807 dan rata-rata sebesar 73,36. Program juga menampilkan jumlah nilai berdasarkan kategori huruf, yaitu:

- **Nilai A: 3**
- **Nilai B: 1**
- **Nilai C: 3**
- **Nilai D: 3**
- **Nilai F: 0**
-

Kesimpulan: Program LetterGrades berhasil mengolah nilai ujian yang dimasukkan pengguna, menghitung total dan rata-rata nilai, serta menampilkan jumlah nilai pada setiap kategori huruf. Program ini menunjukkan penerapan perulangan while, percabangan switch, variabel penghitung, dan operasi aritmetika dalam bahasa pemrograman Java.

Fig. 4.17 LogicalOperators.java

```java
//~~~~ Fig. 4.17 LOGICAL OPERATORS ~~~~~//

public class LogicalOperators {
    public static void main(String[] args) {

        //----- NAMA & NPM -----//
        System.out.println(" --NAMA--------------------------");
        System.out.println("  |  MUHAMMAD FEBRI ARIDAMAYANTO  |");
        System.out.println(" ---------------------------------");

        System.out.println(" --NPM----------");
        System.out.println(" |  262310005  |");
        System.out.println(" ---------------");

        // buat tabel kebenaran untuk operator && (AND kondisional)
        System.out.printf("%s%n%s: %b%n%s: %b%n%s: %b%n%s: %b%n%n",
                "Conditional AND (&&)",
                "false && false", (false && false),
                "false && true", (false && true),
                "true && false", (true && false),
                "true && true", (true && true));

        //buat tabel kebenaran untuk operator || (OR kondisional)
        System.out.printf("%s%n%s: %b%n%s: %b%n%s: %b%n%s: %b%n%n",
                "Conditional OR (||)",
                "false || false", (false || false),
                "false || true", (false || true),
                "true || false", (true || false),
                "true || true", (true || true));

        //create truth table for boolean logical AND (&) operator
        System.out.printf("%s%n%s: %b%n%s: %b%n%s: %b%n%s: %b%n%n",
                "Boolean logical AND (&)",
                "false & false", (false & false),
                "false & true", (false & true),
                "true & false", (true & false),
                "true & true", (true & true));

        //buat tabel kebenaran untuk operator | (OR inklusif logika Boolean)
        System.out.printf("%s%n%s: %b%n%s: %b%n%s: %b%n%s: %b%n%n",
                "Boolean logical inclusive OR (|)",
                "false | false", (false | false),
                "false | true", (false | true),
                "true | false", (true | false),
                "true | true", (true | true));

        //buat tabel kebenaran untuk operator ^ (logika Boolean eksklusif OR)
        System.out.printf("%s%n%s: %b%n%s: %b%n%s: %b%n%s: %b%n%n",
                "Boolean logical exclusive OR (^)",
                "false ^ false", (false ^ false),
                "false ^ true", (false ^ true),
                "true ^ false", (true ^ false),
                "true ^ true", (true ^ true));

        //buat tabel kebenaran untuk operator ! (negasi logika)
        System.out.printf("%s%n%s: %b%n%s: %b%n", "Logical NOT (!)", "!false", (!false), "!true", (!true));
    }
}
```

Output LogicalOperators.java

```
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>  & 'C:\Users\pebri\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pebri\OneDrive\Desktop\Logical Thinking\bin' 'LogicalOperators'
 -NAMA-------------------------
| MUHAMMAD FEBRI ARIDAMAYANTO |
 ---------------------------

 -NPM--------
| 262310005 |
 ------------

Conditional AND (&&)
false && false: false
false && true: false
true && false: false
true && true: true

Conditional OR (||)
false || false: false
false || true: true
true || false: true
true || true: true

Boolean logical AND (&)
false & false: false
false & true: false
true & false: false
true & true: true

Boolean logical inclusive OR (|)
false | false: false
false | true: true
true | false: true
true | true: true

Boolean logical exclusive OR (^)
false ^ false: false
false ^ true: true
true ^ false: true
true ^ true: false

Logical NOT (!)
!false: true
!true: false
PS C:\Users\pebri\OneDrive\Desktop\Logical Thinking>
```

Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java LogicalOperators pada terminal Visual Studio Code. Program ini digunakan untuk menampilkan tabel kebenaran (truth table) dari beberapa operator logika dalam bahasa pemrograman Java, yaitu AND (&&), OR (||), AND logika Boolean (&), OR inklusif (|), XOR (^), dan NOT (!).

Pada awal program, ditampilkan identitas mahasiswa berupa nama MUHAMMAD FEBRI ARIDAMAYANTO dan NPM 262310005. Selanjutnya, program menampilkan hasil operasi logika berdasarkan kombinasi nilai true dan false.

Penjelasan hasil output:
1. Conditional AND (&&)Menghasilkan true hanya jika kedua kondisi bernilai true. Jika salah satu atau kedua kondisi bernilai false, hasilnya adalah false.
2. Conditional OR (||)Menghasilkan true jika minimal salah satu kondisi bernilai true. Hasilnya false jika kedua kondisi bernilai false.
3. Boolean Logical AND (&)Menghasilkan true hanya jika kedua operand bernilai true. Operator ini mengevaluasi kedua operand Boolean.
4. Boolean Logical Inclusive OR (|)Menghasilkan true jika minimal salah satu operand bernilai true, dan false jika keduanya bernilai false.
5. Boolean Logical Exclusive OR (^)Menghasilkan true jika kedua operand memiliki nilai yang berbeda. Jika keduanya sama, hasilnya false.
6. Logical NOT (!)Membalik nilai logika. !false menghasilkan true, sedangkan !true menghasilkan false.

Kesimpulan: Program LogicalOperators berhasil menampilkan hasil berbagai operasi logika berdasarkan kombinasi nilai Boolean. Melalui program ini, dapat dipahami perbedaan fungsi setiap operator logika serta bagaimana operator tersebut menentukan hasil suatu kondisi dalam bahasa pemrograman Java.

Fig. 5.3: MaximumFinder.java

```java
import java.util.Scanner;

public class MaximumFinder {
    public static void main(String[] args) {
        //buat Scanner untuk memperoleh input dari jendela perintah
        Scanner input = new Scanner(System.in);

        //----- NAMA & NPM -----//
        System.out.println(" --NAMA----------------------------");
        System.out.println("   | MUHAMMAD FEBRI ARIDAMAYANTO   |");
        System.out.println(" ---------------------------------");

        System.out.println(" --NPM---------");
        System.out.println(" |  262310005  |");
        System.out.println(" ---------------");

        //meminta dan memasukkan tiga nilai titik mengambang (floating-point)
        System.out.print("Enter three floating-point values separated by spaces: ");
        double number1 = input.nextDouble();
        double number2 = input.nextDouble();
        double number3 = input.nextDouble();

        //
        double result = maximum(number1, number2, number3);

        //
        System.out.println("Maximum is: " + result);

        input.close();
    }

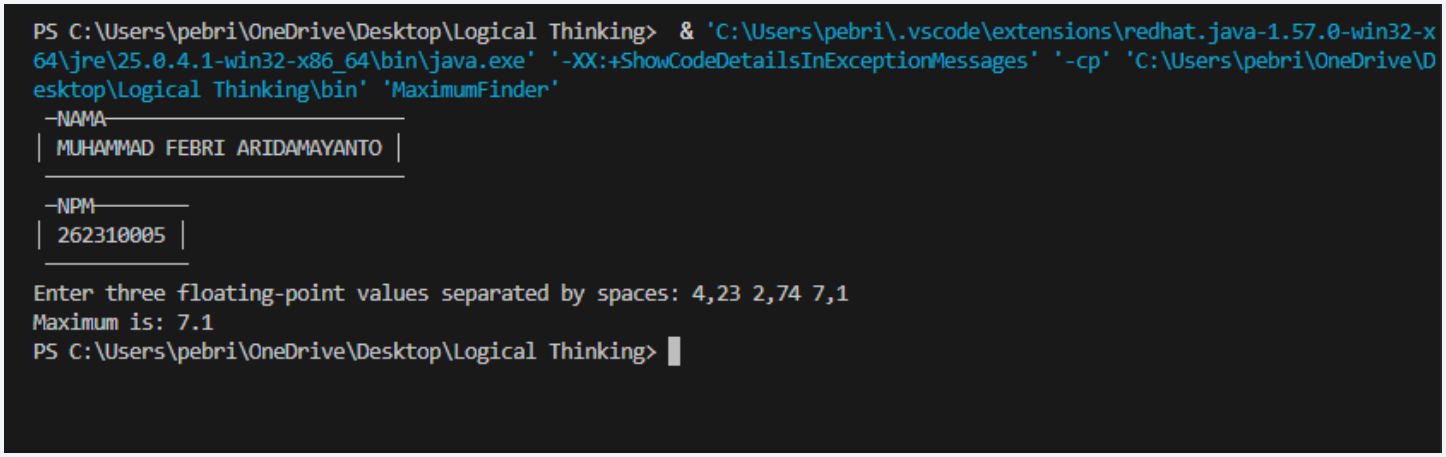
    //
    public static double maximum(double x, double y, double z) {
        double maximumValue = x;

        //
        if (y > maximumValue) {
            maximumValue = y;
        }

        //
        if (z > maximumValue) {
            maximumValue = z;
        }

        return maximumValue;
    }
}
```

Output MaximumFinder.java



Keterangan Output :

Gambar tersebut menunjukkan hasil eksekusi program Java MaximumFinder pada terminal Visual Studio Code. Program ini digunakan untuk menerima tiga bilangan desimal (floating-point) dan menentukan nilai terbesar dari ketiga bilangan tersebut.

Pada awal program, ditampilkan identitas mahasiswa berupa nama MUHAMMAD FEBRI ARIDAMAYANTO dan NPM 262310005. Selanjutnya, pengguna memasukkan tiga bilangan, yaitu 4,23, 2,74, dan 7,1.

Setelah ketiga bilangan dimasukkan, program membandingkan nilai-nilai tersebut dan menampilkan hasil "Maximum is: 7.1". Hasil ini menunjukkan bahwa angka 7,1 merupakan nilai terbesar dibandingkan dengan 4,23 dan 2,74.

Kesimpulan: Program MaximumFinder berhasil menentukan nilai terbesar dari tiga bilangan desimal yang dimasukkan oleh pengguna. Program ini menunjukkan penerapan input data, penggunaan tipe data bilangan desimal, serta operasi perbandingan untuk mencari nilai maksimum dalam bahasa pemrograman Java.